BLOG

R and openMP: boosting compiled code on multi-core cpu-s (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/)

JULY 26, 2016 *BY* OTT TOOMET (HTTP://WWW.PARALLELR.COM/AUTHOR/OTT-TOOMET/)

Share This:

INTRODUCTION

Sometimes you just need more speed. And sometime plain R does not provide it.

This article is about boosting your R code with C++ and openMP. OpenMP is a parallel processing framework for shared memory systems. This is an excellent way to use all the cpu cores that are sitting, and often just idling, in any modern desktop and laptop. Below, I will take a simple, even trivial problem—ML estimation of normal distribution parameters—and solve it first in R, thereafter I write the likelihood function in standard single-threaded C++, and finally in parallel using C++ and openMP. Obviously, there are easier ways to find sample mean and variance but this is not the point. Read it, and try to write your own openMP program that does something useful!

R FOR SIMPLICITY

Assume we have a sample of random normals and let's estimate the parameters (mean and standard deviation) by Maximum Likelihood (ML). We start with pure R. The log-likelihood function may look like this:

```
llR <- function(par, x) {
   mu <- par[1]
   sigma <- par[2]
   sum(-1/2*log(2*pi) - log(sigma) - 1/2*((x - mu)^2)/sigma^2)
}
```

Note that this code is fully vectorized (`(x–mu)` is written with no explicit loop) and hence very fast. Obviously, this is a trivial example, but it is easy to understand and parallelize.

Now generate some data

```
x <- rnorm(1e6)
```

NEWSLETTER

Email address:

Your email address

Subscribe

SEARCH

Search

CATEGORIES

Accelerators (http://www.parallelr.com/category/mic-fpga-asic/)

General (http://www.parallelr.com/category/generinforamion/)

GPGPU (http://www.parallelr.com/category/mic-fpga-asic/general-purpose-gpu-computing/)

Intel Xeon Phi (http://www.parallelr.com/category/mic-fpga-asic/intel-xeon-phi/)

MPI (http://www.parallelr.com/category/openr message-passing-interface/)

MultiCores (http://www.parallelr.com/category/openr tbb-pthread-multithreads/)

and start values, a bit off to give the computer more work:

```
start <- c(1,1)
```

Estimate it (using maxLik package):

```
library(maxLik)
system.time(m <- maxLik(llR, start=start, x=x))
```

```
   user system elapsed
  2.740  0.184   2.931
```

```
summary(m)
```

```
--------------------------------------------
Maximum Likelihood estimation
Newton-Raphson maximisation, 6 iterations
Return code 2: successive function values within tolerance limit
Log-Likelihood: -1419125
2 free parameters
Estimates:
Estimate Std. error t value Pr(> t)
[1,] 0.0010318 0.0010001 1.032 0.302
[2,] 1.0001867 0.0007072 1414.236 <2e-16 ***
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
--------------------------------------------
```

The code runs 2.6s on an i5-2450M laptop using a single cpu core.

First, let's squeeze more out of the R code. Despite being vectorized, the function can be improved by moving the repeated calculations out of the (vectorized) loop. We can re-write it as:

```
llROpt <- function(par, x) {
   mu <- par[1]
   sigma <- par[2]
   N <- length(x)
   -N*(0.5*log(2*pi) + log(sigma)) - 0.5*sum((x - mu)^2)/sigma^2
}
```

Now only `(x - mu)^2` is computed as vectors. Run it:

```
library(maxLik)
system.time(m <- maxLik(llROpt, start=start, x=x))
```

```
   user  system elapsed
  0.816   0.000   0.818
```

You see—just a simple optimization gave a more than three–fold speed improvement! I don't report the results any more, as those are virtually identical.

C FOR SPEED

Now let's implement the same function in C++. R itself is written in C, however there is an excellent library, Rcpp, that makes integrating R and C++ code very easy. It is beyond the scope of this post to teach readers C and explain the differences between C and C++. But remember that Rcpp (and hence C++) offers a substantially easier interface for exchanging data between R and compiled code than the default R API. Let's save the log-likelihood function in file loglik.cpp. It might look like this:

```
#include <cmath>
#include <Rcpp.h>

using namespace Rcpp;

RcppExport SEXP loglik(SEXP s_beta, SEXP s_x) {
    NumericVector x(s_x);
    NumericVector beta(s_beta);
    // make Rcpp vector out of R SEXP
```

Performance Optimizaiton (http://www.parallelr.com/category/cache efficiency/)

Vectorization (http://www.parallelr.com/category/vecto

RECENT POSTS

R benchmark for High-Performance Analytics and Computing (II): GPU Packages (http://www.parallelr.com/r-hpac-benchmark-analysis-gpu/)

Parallel Computation with R and XGBoost (http://www.parallelr.com/parallel-computation-with-r-and-xgboost/)

R with Parallel Computing from User Perspectives (http://www.parallelr.com/r-with-parallel-computing/)

R and openMP: boosting compiled code on multi-core cpu-s (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/)

R for Deep Learning (III): CUDA and MultiGPUs Acceleration (http://www.parallelr.com/r-dnn-cuda-multigpu/)

BLOG ROLL

R-bloggers (http://www.r-bloggers.com/)

Simple Statistics (http://simplystatistics.org/)

Jun's Blog (http://junma5.weebly.com/data-blog/)

```
    double mu = beta[0];
    // first element is 0 in C++
    double sigma = beta[1];
    double ll = 0;
    for(int i = 0; i < x.length(); i++) {
        ll -= (x[i] - mu)*(x[i] - mu);
    }
    ll *= 0.5/sigma/sigma;
    ll -= (0.5*log(2*M_PI) + log(sigma))*x.length();
    NumericVector result(1, ll);
    // create 'numeric' vector of length 1, filled with
    // ll values
    return result;
}
```

The function takes two parameters, `s_beta` and `s_x`. These are passed as R general vectors, denoted `SEXP` in C. As SEXP-s are complicated to handle, the following two lines transform those to 'NumericVector's, essentially equivalent to R 'numeric()'. The following code is easy to understand. We loop over all iterations and add (x[i] – mu)^2. Loops are cheap in C++. Afterwards, we add the constant terms only once. Note that unlike R, indices in C++ start from zero.

This program must be compiled first. Normally the command

```
R CMD SHLIB loglik.cpp
```

takes care of all the R-specific dependencies, and if everything goes well, it results in the DLL file. Rcpp requires additional include files which must be specified when compiling, the location of which can be queried with `Rcpp:::CxxFlags()` command in R, or as a bash one-liner, one may compile with

```
PKG_CXXFLAGS=$(echo 'Rcpp:::CxxFlags()'| R --vanilla --slave) R CMD SHLIB l
```

Now we have to create the R-side of the log-likelihood function. It may look like this:

```
llc <- function(par, x) {
   library(Rcpp)
   dyn.load("loglik.so") # extension '.so' is platform-specific!
   res <- .Call("loglik", par, x)
   res
}
```

It takes arguments 'par' and 'x', and passes these down to the DLL. Before we can invoke (`.Call`) the compiled code, we must load the DLL. You may need to adjust the exact name according to your platform here. Note also that I haven't introduced any security checks neither at the R nor the C++ side. This is a quick recipe for crashing your session, but let's avoid it here in order to keep the code simple.

Now let's run it:

```
system.time(m <- maxLik(llc, start=start, x=x))
```

```
   user  system elapsed
  0.896   0.020   0.913
```

The C code runs almost exactly as fast as the optimized R. In case of well vectorized computations, there seems to be little scope for improving the speed by switching to C.

PARALLELIZING THE CODE ON MULTICORE CPUS

Now it is time to write a parallel version of the program. Take the C++ version as the point of departure and re-write it like this:

```
#include <cmath>
#include <Rcpp.h>
#include <omp.h>

using namespace Rcpp;

```

DISCLAIMER

1) This is a personal weblog.
2) The opinions expressed here represent my own and not those of my employer.
3) The opinions expressed by the ParallelR Bloggers and those providing comments are theirs alone and do not reflect the opinions of ParallelR.

```
RcppExport SEXP loglik_MP(SEXP s_beta, SEXP s_x, SEXP s_nCpu) {
    NumericVector x(s_x);
    NumericVector beta(s_beta);
    int n_cpu = IntegerVector(s_nCpu)[0];
    double mu = beta[0];
    double sigma = beta[1];
    double ll = 0;
    omp_set_dynamic(0);         // Explicitly disable dynamic teams
    omp_set_num_threads(n_cpu); // Use n_cpu threads for all
                                // consecutive parallel regions
#pragma omp parallel
    {
        double ll_thread = 0;
#pragma omp for
        for(int i = 0; i < x.length(); i++) {
            ll_thread -= (x[i] - mu)*(x[i] - mu);
        }
#pragma omp critical
        {
            ll += ll_thread;
        }
    }
    ll *= 0.5/sigma/sigma;
    ll -= (0.5*log(2*M_PI) + log(sigma))*x.length();
    NumericVector result(1, ll);
    return result;
}
```

The code structure is rather similar to the previous example. The most notable novelties are the '#pragma omp' directives. These tell the compiler to insert parallelized code here. Not all compilers understand it, and others may need special flags, such as `-fopenmp` in case of gcc, to enable openMP support. Otherwise, gcc just happily ignores the directives and you will get a single-threaded application. The likelihood function also includes the argument *n_Cpu*, and the commands `omp_set_dynamic(0)` and `omp_set_num_threads(n_cpu)`. This allows to manipulate the number of threads explicitly, it is usually not necessary in the production code.

For compiling the program, we can add `-fopenmp` to our one-liner above:

```
PKG_CXXFLAGS="$(echo 'Rcpp:::CxxFlags()'| R --vanilla --slave) -fopenmp" R
```

assuming it was saved in "loglikMP.cpp". But now you should seriously consider writing a makefile instead.

We use three openMP directives here:

```
#pragma omp parallel
{
/* code block */
}
```

This is the most important omp directive. It forces the code block to be run in multiple threads, by all threads simultaneously. In particular, variable *ll_thread* is declared in all threads separately and is thus a thread-specific variable. As OMP is a shared-memory parallel framework, all data declared before *#pragma omp parallel* is accessible by all threads. This is very convenient as long as we only read it.

The last directive is closely related:

```
#pragma omp critical
{
/* code block */
}
```

This denotes a piece of threaded code that must be run by only one thread simultaneously. In the example above all threads execute `ll += ll_thread`, but only one at a time, waiting for the previous thread to finish if necessary. This is because now we are writing to shared memory: variable *ll* is defined before we split the code into threads. Allowing multiple threads to simultaneously write in the same shared variable almost always leads to trouble.

Finally,

```
#pragma omp for
for(...) { /* code block */ }
```

splits the for loop between threads in a way that each thread will only go through a fraction of the full loop. For instance, in our case the full loop goes over 1M observations, but in case of 8 threads, each will receive only 125k. As the compiler has to generate code for this type of loop sharing, parallel loops are less flexible than ordinary single-threaded loops. For many data types, summing the thread–specific values we did with `#pragma omp critical` can be achieved directly in the loop by specifying `#pragma omp parallel for reduction(+:ll)` instead.

As all the parallel work is done at C level, the R code remains essentially unchanged. We may write the corresponding loglik function as

```
llcMP <- function(par, nCpu=1, x) {
   library(Rcpp)
   dyn.load("loglikMP.so")
   res <- .Call("loglik_MP", par, x, as.integer(nCpu))
   res
}
```

How fast is this?

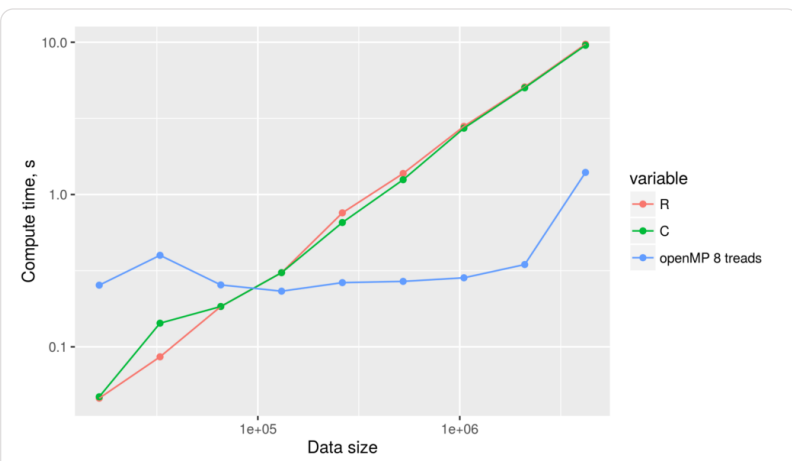
```
system.time(m <- maxLik(llcMP, start=start, nCpu=4, x=x))
```

```
   user  system elapsed
  0.732   0.016   0.203
```

On 2-core/4-thread cpu, we got a more than four–fold speed boost. This is impressive, given the cpu does have 2 complete cores only. Obviously, the performance improvement depends on the task. This particular problem is embarrasingly parallel, the threads can work completely independent of each other.

TIMING EXAMPLES

As an extended timing example, we run all the (optimized) examples above using a Xeon-L5420 cpu with 8 cores, single thread per core. The figure below depicts the compute time for single-threaded R and C++ code, and for C++/openMP code with 8 threads, as a function of data size.



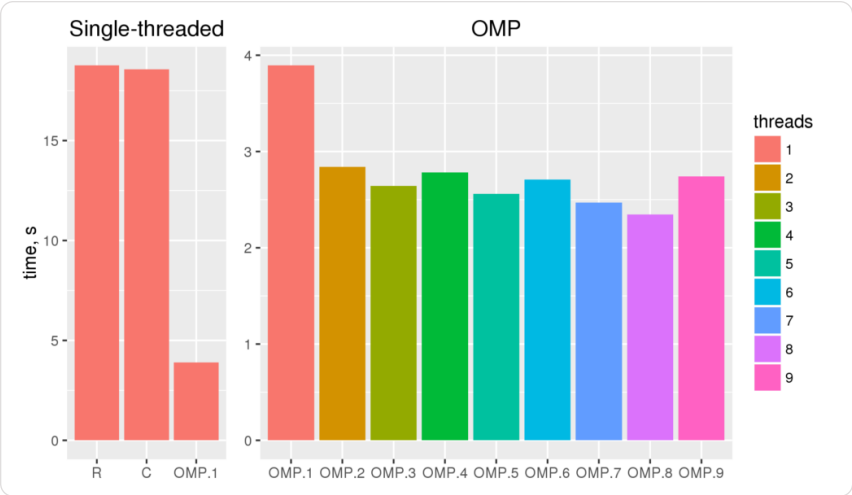
The figure reveals several facts. First, for non-parallelized code we can see that

1. Optimized R and C++ code are of virtually identical speed.
2. compute time grows linearily in data size.

For openMP code the figure tells

3. openMP with 8 threads is substantially slower for data size less than about 100k. For larger data, multi-threaded approach is clearly faster.
4. openMP execution time is almost constant for data size up to 4M. For larger data vectors, it increases linearily. This suggests that for smaller data size, openMP execution time is dominated by thread creation and management overheads, not by computations.

Finally, let's compare the computation times for different number of threads for 8M data size.



The figure shows the run time for single threaded versions of the code (R and C), and multi-threaded openMP versions with 1 to 9 threads (OMP.1 to OMP.9).

1. More cpus give us shorter execution times. 1-thread OMP will run almost 1.7 times slower than 8-threaded version (3.9 and 2.3 s respectively).
2. The gain of more cpu cores working on the problem levels off quickly. Little noticeable gain is visible for more than 3 cores. It indicates that the calculations are only partly limited by computing-power. Another major bottleneck may be memory speed.
3. Last, and most strikingly, even the single threaded OMP version of the code is 4.8 times faster than single-threaded C++ version with no OMP (18.6 and 3.9 s respectively)! This is a feature of the particular task, the compiler and the processor architecture. OMP parallel for–loops allow the compiler to deduce that the loops are in fact independent, and use faster SSE instruction set. This substantially boosts the speed but requires more memory bandwidth.

CONCLUSION

With the examples above I wanted to show that for many tasks, openMP is not hard to use. If you know some C++, parallelizing your code may be quite easy. True, the examples above are easy to parallelize at R-level as well, but there are many tasks where this is not true.

Obviously, in the text above I just scratched the surface of openMP. If you consider using it, there are many excellent sources on the web. Take a look!

I am grateful to Peng Zhao for explaining the parallel loops and SSE instruction set.

← R for Deep Learning (III): CUDA and MultiGPUs Acceleration (http://www.parallelr.com/r-dnn-cuda-multigpu/)
R with Parallel Computing from User Perspectives → (http://www.parallelr.com/r-with-parallel-computing/)

9 COMMENTS

Yixuan Qiu (http://statr.me/) says:

July 27, 2016 at 12:50 am (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-288)

Thanks for the nice post.

One quick comment: As far as I know the accessor of `Rcpp::NumericVector` is not thead-safe (see the discussion in http://rcppcore.github.io/RcppParallel/#thread_safety (http://rcppcore.github.io/RcppParallel/#thread_safety)), so in the OpenMP example, it is better to first get the pointer of x, and then use the pointer in the parallelized for loop:

```
double* xpt = x.begin();
const int xlen = x.length();
// …
#pragma omp for
for(int i = 0; i < xlen; i++) {
ll_thread -= (xptr[i] – mu)*(xptr[i] – mu);
}
```

In a word, it's worth mentioning that no R/Rcpp API should be used inside OpenMP.

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=288#respond)

Ott Toomet says:

July 27, 2016 at 2:39 am (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-289)

Thank you for your comment.

I Agree. But as I am only reading from it in parallel it works here. It does not work for writing.

There are several alternatives, your pointer example is one, but you may also convert it to a thread-safe vector (STL vector, for instance), or maybe use a thread-safe library that supports linear algebra.

I thought about mentioning this, but the post is quite long even now…

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=289#respond)

Bjørn-Helge Mevik says:

July 30, 2016 at 2:47 pm (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-301)

Thanks for the post! I found it a gentle, but clear introduction to the subject. It was an eye opener that OpenMP with a single thread could be more efficient due to the compiler hints.

One small typo: In loglik.cpp, the function is called "loglik_opt", while llc() uses .Call("loglik", …).

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=301#respond)

Ott Toomet (http://www.obs.ee/~siim) says:

July 30, 2016 at 6:26 pm (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-302)

Thanks!
Corrected now.

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=302#respond)

Mark van der Loo (http://www.markvanderloo.eu) says:

August 4, 2016 at 7:40 am (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-306)

Perhaps good to note that if you use openMP in a CRAN package, you are expected to make the number of nodes requested configurable by the user, e.g.

```
SEXP myfun(SEXP nthreads_){
int nthreads = INTEGER(nthreads_)[0]
#pragma omp parallel num_threads(nthreads)
{
// insert parallel code here
}

// …

}
```

See the famous WRE: https://cran.r-project.org/doc/manuals/r-release/R-exts.html#OpenMP-support (https://cran.r-project.org/doc/manuals/r-release/R-exts.html#OpenMP-support)

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=306#respond)

Ott Toomet (http://www.obs.ee/~siim) says:

August 4, 2016 at 3:42 pm (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-307)

Thanks, good to know.

The OMP stuff in WRE is relatively recent? I cannot remember seeing it some time ago when reading about the compiled code..

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=307#respond)

Mark van der Loo says:

August 4, 2016 at 8:09 pm (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-310)

I think it's been there at least since about late 2013, when I started parallelizing my stringdist pkg

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=310#respond)

Jinzhu says:

March 5, 2017 at 3:37 am (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-2055)

Thanks for the nice post.
I used " #pragma omp parallel " and my function can execute with multi-thread. However, when I try to build a package, Rstudio tell me " #pragma omp parallel " was ignored.
Can you help me fix this problem?

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=2055#respond)

Ott Toomet (http://www.obs.ee/~siim) says:

March 5, 2017 at 6:11 am (http://www.parallelr.com/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/#comment-2056)

Hi Jinzhu,
thank you for your comment. Can you tell what exactly did you do? Which platform do you use (windows/mac/linux/…?)

Strictly speaking, Rstudio has no clue about #pragmas. It must be your C compiler that complains. I guess you have to either use the correct flags for the compiler (the case with gcc), or perhaps a different compiler (the case with clang, afaik). But there is little I can say unless I know better what exactly are you doing.

Reply (/r-and-openmp-boosting-compiled-code-on-multi-core-cpu-s/?replytocom=2056#respond)

LEAVE A REPLY

Your email address will not be published. Required fields are marked *

Comment

Name *

Email *

Website

Notify me of followup comments via e-mail. You can also subscribe (http://www.parallelr.com/comment-subscriptions/?srp=998&srk=7c88bab4554719eb6eda8e5de4a43dff&sra=s&srsrc=f) without commenting.

POST COMMENT

Copyright © ParallelR 2019